From RNNs to BERT: A Mathematical Journey

Understanding Encoder-Decoder, Attention, and Contextual Embeddings

A Guide to NLP Architectures

Contents

Introduction: The Quest for Understanding Language

Welcome to a deep dive into Natural Language Processing (NLP). Our goal is to understand how we moved from simple mechanical translations to systems that seem to "understand" context, nuance, and ambiguity. We will trace the evolution of three major architectures:

1. **Recurrent Neural Networks (RNNs):** The sequential processors.
2. **LSTMs:** The memory masters.
3. **Transformers (and BERT):** The parallel revolutionaries.

We will use concrete examples, "back-of-the-envelope" math, and rich analogies to make these complex topics intuitive.

1 The Translation Challenge & The Encoder-Decoder Intuition

Imagine you are a translator. Someone hands you a sentence in French: *"Le chat est noir"*. To translate this into English, you don't just swap words one-by-one.

- *"Le"* $\rightarrow$ *"The"*
- *"chat"* $\rightarrow$ *"cat"*

This "word-for-word" approach fails for complex sentences like *"Je t'aime"* ("I you love" vs "I love you").

Instead, you read the **entire** French sentence first. You build a mental picture—a "thought"—of a black cat. Then, you forget the French words and describe that "thought" in English.

1.1 The Architecture: The "Zip File" Analogy

In Deep Learning, this is the **Encoder-Decoder** architecture.

1. **The Encoder:** Reads the input and compresses it into a **Context Vector**.
    - *Analogy:* Think of this as creating a `.zip` file of a text document. You compress all the information into a single, dense binary blob.
2. **The Decoder:** Unzips this vector to generate the output.

The Bottleneck Problem: The Encoder forces the entire meaning of a sentence—whether it's 5 words or 500 words—into a fixed-size vector (e.g., 256 numbers). If the sentence is too long, details get lost in compression. This is the fundamental flaw we will eventually solve with Attention.

Let's visualize this high-level flow:

The magic lies in how we mathematically represent words and how we process them sequentially. To do this, we need a special type of neural network: the **Recurrent Neural Network (RNN)**.

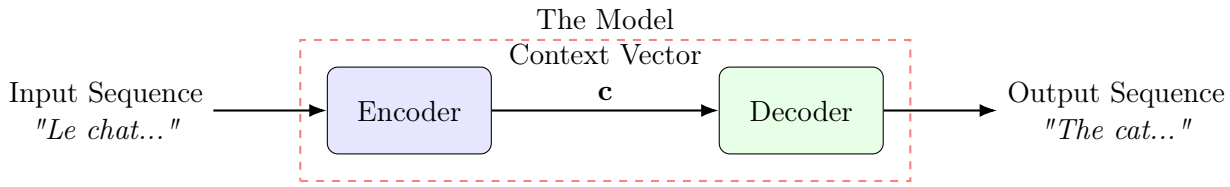

Figure 1: The High-Level Encoder-Decoder Architecture. The Encoder compresses the input into a Context Vector, which the Decoder uses to generate the output.

2 The Engine: Recurrent Neural Networks (RNNs)

Standard neural networks have a limitation: they expect a fixed-size input. But sentences vary in length. *"Hello"* is one word; *"The quick brown fox..."* is longer. To handle this, we need a network that has a loop, allowing it to process information step-by-step while maintaining a "memory" of what it has seen so far. This is the **Recurrent Neural Network (RNN)**.

2.1 A Concrete Example: "I don't like NLP"

Let's demystify the math with a story. Imagine our RNN is reading the sentence: **"I don't like NLP"**. To a computer, these words are just vectors. Let's assign a simple 2-dimensional vector to each word for our example:

$$\text{"I"} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \text{"don't"} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \text{"like"} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \text{"NLP"} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

2.2 The Hidden State: The "Color Mixing" Analogy

The core of the RNN is the **Hidden State**, denoted as h. It represents the network's current understanding of the sentence.

Think of the Hidden State as a bucket of water.

- The water starts clear ($h_0 = 0$).
- Each word is a drop of colored dye.
- When we read "I" (Red dye), the water turns Red (h_1).
- When we read "don't" (Blue dye), we pour it into the Red water. It becomes Purple (h_2).
- The final color depends on *all* previous drops.

Mathematically, we update the state using this formula:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \tag{1}$$

Deconstructing the Matrices: Assume our hidden state size is 2 and input size is 2.

- W_h (2×2): Determines how much of the *old memory* to keep.
- W_x (2×2): Determines how much of the *new word* to add.

- tanh: Squashes values to $[-1, 1]$. Without this, repeated matrix multiplications would make numbers explode to infinity (like a microphone feedback loop).

2.3 Let's Crunch the Numbers

Let's initialize our matrices with simple numbers so we can see the math in action. Assume the initial hidden state $h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ (empty memory).

Let our weights be:

$$W_h = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad W_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Step 1: Processing "I" ($x_1 = [1, 0]^T$)

$$h_1 = \tanh\left(\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right)$$

$$h_1 = \tanh\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) = \tanh\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) \approx \begin{bmatrix} 0.76 \\ 0 \end{bmatrix}$$

Now, our memory (h_1) holds the concept of "I".

Step 2: Processing "don't" ($x_2 = [0, -1]^T$)

$$h_2 = \tanh(W_h h_1 + W_x x_2)$$

$$h_2 = \tanh\left(\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.76 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right)$$

$$h_2 = \tanh\left(\begin{bmatrix} 0.38 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right) = \tanh\left(\begin{bmatrix} 0.38 \\ -1 \end{bmatrix} \right) \approx \begin{bmatrix} 0.36 \\ -0.76 \end{bmatrix}$$

Notice what happened! The new state h_2 combines information from "I" (the positive first dimension) and "don't" (the negative second dimension). The RNN is blending the meanings together!

2.4 Visualizing the Unrolled RNN

This process continues until the end of the sentence. We can visualize this "unrolling" over time:

3 The Problem: Context Loss (The Telephone Game)

While RNNs are great in theory, they suffer from a major problem in practice: **Context Loss** (mathematically known as the Vanishing Gradient problem).

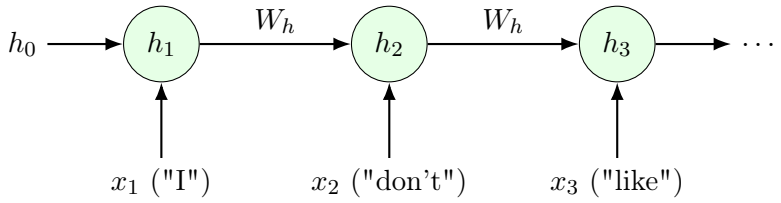

Figure 2: The Unrolled RNN. Information flows from left to right, accumulating context.

Imagine playing the "Telephone Game" where a message is whispered from person to person. By the time it reaches the 20th person, the original message is often lost or distorted. In an RNN, as the information (h_t) passes through many time steps (multiplications by W_h), the signal from the beginning of the sentence gets weaker and weaker.

If we have a long sentence like: *"The **cats**, who were playing in the garden all day long ..., **are** hungry."*

The network needs to remember "**cats**" (plural) to predict "**are**" (plural) at the end. But in a standard RNN, by the time it gets to the end, it might have forgotten "cats" and predict "is".

3.1 The Solution: Long Short-Term Memory (LSTM)

To fix this, we need a smarter neuron. Enter the **LSTM**.

The Conveyor Belt Analogy: Think of the standard RNN as a game of "Telephone" (information degrades). Think of the LSTM as a **Conveyor Belt** (the Cell State C_t).

- Packages (Information) are placed on the belt at the start.
- As the belt moves, we have workers (Gates) standing alongside.
- One worker might remove a package (**Forget Gate**).
- Another might add a new package (**Input Gate**).
- Crucially, if the workers do nothing, the package travels safely to the end *untouched*.

This direct path allows gradients to flow back easily, solving the Vanishing Gradient problem.

The Mathematical Gates: The gates are neural networks themselves, outputting values between 0 (Close gate) and 1 (Open gate) using the Sigmoid function (σ).

1. **Forget Gate (f_t):** Decides what to keep from C_{t-1}.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input Gate (i_t):** Decides what new info ($\tilde{C}_t$) to add.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3. **Cell Update:** The old conveyor belt C_{t-1} is modified:

$$C_t = (f_t \times C_{t-1}) + (i_t \times \tilde{C}_t)$$

(Forget some old stuff + Add some new stuff)

4. **Output Gate (o_t):** Decides the new hidden state h_t.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \times \tanh(C_t)$$

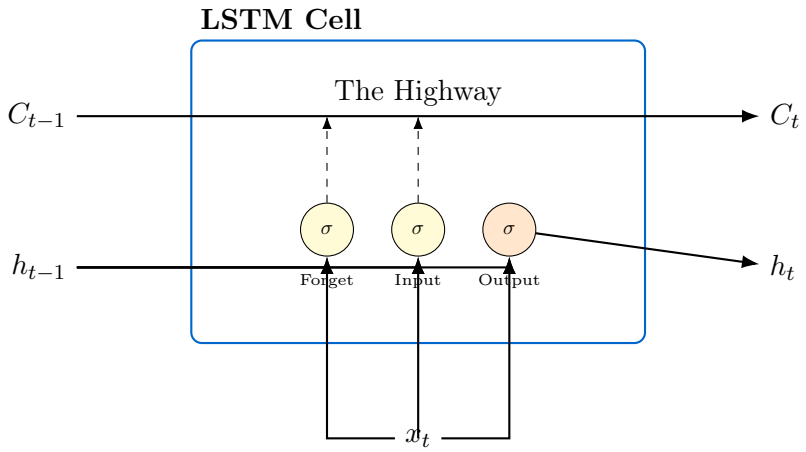

Figure 3: Conceptual Diagram of an LSTM. The "Highway" (C_t) allows information to flow unimpeded, while gates regulate updates.

LSTMs became the standard for years because they solved the vanishing gradient problem. However, as sentences got even longer (paragraphs, documents), a new problem emerged: **The Bottleneck**.

4 The Attention Mechanism

The Encoder-Decoder architecture we saw earlier had a flaw: it forced the Encoder to compress the entire meaning of a long sentence into a **single fixed-size vector**. It's like asking you to read a whole book and then summarize it in exactly 5 numbers. You will inevitably lose details.

4.1 Intuition: The "Spotlight" Analogy

Attention changes the game. Instead of relying on just the final "summary" vector, the Decoder is allowed to **look back** at *all* the hidden states of the Encoder at every step.

Imagine a dark room (The Encoder's memory).

- You (The Decoder) have a flashlight (The Attention Mechanism).
- When you want to translate "noir", you shine your flashlight on the French word "noir".

- You focus 90% of the light on "noir" and maybe 10% on "chat" (because the gender of the cat matters).
- You ignore "Le" (0

This is called **Soft Alignment**. Unlike "Hard Alignment" (where you strictly pick one word), Soft Alignment allows the model to learn to focus on multiple relevant words at once (e.g., "Poste" in French could mean "Mail" or "Job" depending on the context, so the model looks at surrounding words too).

4.2 Mathematical Walkthrough: Calculating Attention

Let's do exactly what we did for RNNs: use real numbers. Suppose the Decoder is trying to translate a word and its current hidden state is s_t. The Encoder has produced two hidden states (h_1, h_2) representing the source words "Bank" and "River".

Let's define 2D vectors:

$$s_t = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ (Decoder State)}, \quad h_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ ("Bank")}, \quad h_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \text{ ("River")}$$

Notice that s_t aligns better with the first dimension of h_1.

Step 1: Calculate Alignment Scores (e) We calculate the Dot Product to measure similarity. *Mathematical Intuition:* Geometrically, the dot product checks if two vectors point in the same direction. If they do, the result is large (high attention). If they are perpendicular (unrelated), the result is 0.

$$e_1 = s_t \cdot h_1 = (1 \times 1) + (0 \times 1) = 1$$

$$e_2 = s_t \cdot h_2 = (1 \times 0) + (0 \times 1) = 0$$

The score for "Bank" (1) is higher than "River" (0).

Step 2: Calculate Attention Weights (α) We apply the **Softmax** function to turn scores into probabilities (percentages).

$$\text{Sum of Exponentials} = e^1 + e^0 \approx 2.718 + 1 = 3.718$$

$$\alpha_1 = \frac{e^1}{3.718} \approx 0.73 \quad (73\%)$$

$$\alpha_2 = \frac{e^0}{3.718} \approx 0.27 \quad (27\%)$$

The model decides to pay 73% attention to "Bank" and 27% to "River".

Step 3: Compute the Context Vector (c_t) Finally, we compute the weighted sum. This

effectively "filters" the information.

$$c_t = 0.73h_1 + 0.27h_2$$

$$c_t = 0.73 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 0.27 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.73 \\ 0.73 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.27 \end{bmatrix} = \begin{bmatrix} 0.73 \\ 1.00 \end{bmatrix}$$

The resulting context vector c_t is a blend, but it is heavily dominated by the features of "Bank". This vector is what the decoder actually uses to predict the next word.

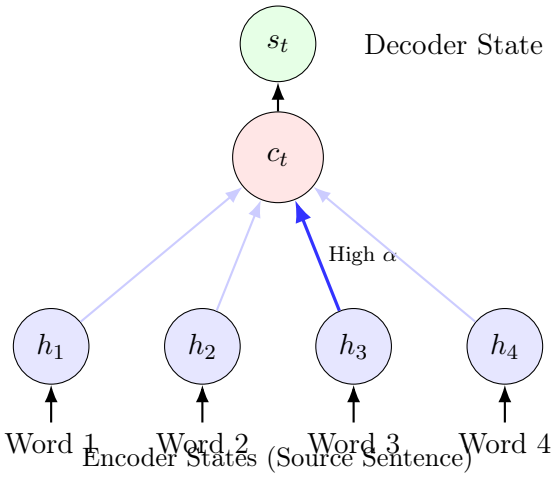

Figure 4: The Attention Mechanism. The Decoder calculates a weighted sum (c_t) of all Encoder states, focusing more on relevant words (e.g., h_3).

Attention revolutionized NLP. But we were still using static vectors for words (like our example where "I" was always $[1, 0]$).

5 The Transformer Revolution

In 2017, a paper titled *"Attention is All You Need"* proposed a radical idea: **Throw away the RNNs**. RNNs are slow because they are sequential (you can't calculate step 10 before step 9). The Transformer relies *entirely* on Attention, allowing it to process the whole sentence at once (parallelization).

5.1 Self-Attention: Queries, Keys, and Values

In the Encoder-Decoder attention, the **Decoder** looked at the **Encoder**. In **Self-Attention**, the sentence looks at *itself*. Every word asks: "How does every other word in this sentence relate to me?"

To do this, we create three new vectors for every word:

- **Query (Q):** What the word is looking for.
- **Key (K):** What the word announces to others (like a label).
- **Value (V):** The actual meaning content of the word.

The Library Analogy: Imagine searching for a book in a library.

- Your search term is the **Query**.
- The title on the book spine is the **Key**.
- If the Query matches the Key (high score), you take the book's content, which is the **Value**.

5.2 The Transformer Math

The math looks very similar to what we just did, but in matrix form:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

Here is what is happening:

1. QK^T: Calculate the dot product (similarity) between every Query and every Key.
2. $\sqrt{d_k}$: We divide by the square root of the vector dimension to keep numbers stable (scaling).
3. softmax: Convert scores to probabilities (attention weights).
4. $\ldots V$: Multiply by the Values to get the final weighted sum.

5.3 Multi-Head Attention: The "Committee of Advisors"

Why do we need just one attention score? A sentence is complex. Consider: *"The animal didn't cross the street because it was too tired."*

- One perspective links "it" to "animal" (Subject resolution).
- Another links "it" to "tired" (Adjective description).

If we only have one set of Q, K, V, the model has to average these meanings out.

Analogy: Imagine the President (the model) has a **Committee of Advisors**.

- Advisor 1 (Grammar Head): Focuses on subject-verb agreement.
- Advisor 2 (Context Head): Focuses on linking pronouns ("it") to nouns ("animal").
- Advisor 3 (Tone Head): Focuses on the emotional sentiment.

Mathematically, we run the attention mechanism multiple times (e.g., 8 heads) in parallel.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{3}$$

We then concatenate their advice and pass it through a linear layer (W^O) to synthesize the final decision.

5.4 Feed-Forward Networks: The "Thinking" Step

After the attention heads gather information (the "Lookup" phase), we need to process it. The **Position-wise Feed-Forward Network** is applied to each word independently.

- It expands the dimension (e.g., from 512 to 2048) and then compresses it back.
- **Intuition:** This is where the model "thinks" about what it just found in the library. It processes the value extracted by attention to build a deeper representation.

5.5 Masking: Hiding the Future

There are two types of masks used in Transformers:

1. **Padding Mask:** Sentences have different lengths. We add "pad" tokens to make them equal. The mask ensures the attention mechanism ignores these dummy tokens (sets their score to $-\infty$).
2. **Look-Ahead Mask (Decoder only):** When the decoder is generating the 3rd word, it shouldn't be allowed to "cheat" and see the 4th word of the training sentence. We mask out all future positions.

5.6 Add & Norm: Stability and Depth

Deep networks are notoriously hard to train due to vanishing gradients. The Transformer adds two crucial tricks after every attention and feed-forward block:

1. Residual Connections (The "Add" part): Instead of just passing x through a layer to get $f(x)$, we output $x + f(x)$. This creates a "highway" for gradients to flow backward without diminishing. If the layer $f(x)$ is useless, the model can just ignore it and pass x through.

2. Layer Normalization (The "Norm" part): We standardize the numbers (subtract mean, divide by variance) within each layer. This keeps the values stable and prevents them from exploding or shrinking too much.

$$\text{LayerOutput} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

5.7 Positional Encodings: The "Clock Hands" Analogy

If we remove RNNs, the model loses the sense of order. It doesn't know that "Man bites dog" is different from "Dog bites man" because it sees all words at the same time. To fix this, we add **Positional Encodings**—special vectors added to the input embeddings.

But we don't just add numbers like 1, 2, 3 (because for long sentences, numbers get too big). Instead, we use Sine and Cosine functions of different frequencies.

Analogy: Think of a clock with multiple hands moving at different speeds.

- The "Second hand" moves fast (High frequency).

- The "Hour hand" moves slow (Low frequency).

By looking at the combination of hands, you can tell the exact time (position) uniquely, even if the time is very large. Mathematically, the vector has this pattern:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

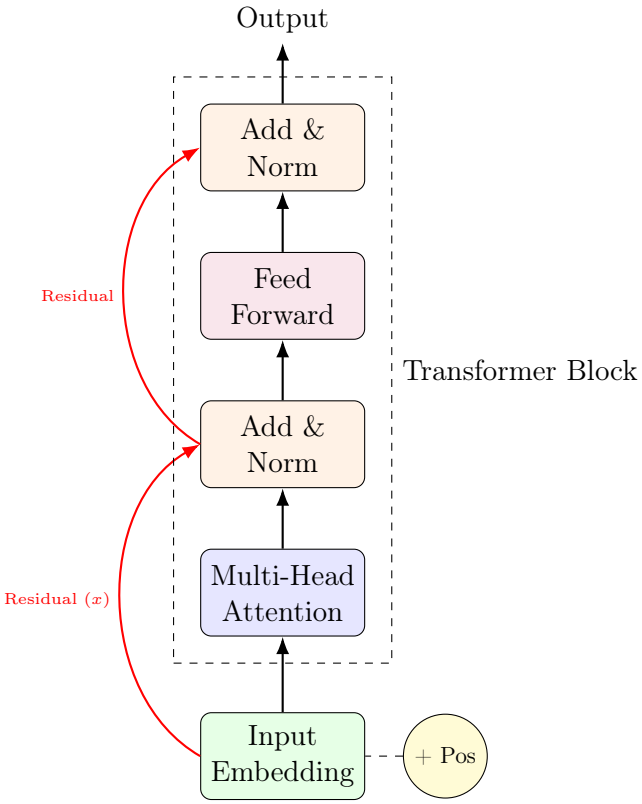

Figure 5: A Detailed Transformer Block. Notice the Residual connections jumping over the layers to the "Add & Norm" blocks.

With the Transformer, we could train massive models on massive datasets. This leads us to the final piece of the puzzle.

6 Contextual Embeddings and BERT

Now that we understand the Transformer, we can finally meet **BERT** (Bidirectional Encoder Representations from Transformers). Simply put, BERT is a **stack of Transformer Encoders**. It was the "ImageNet moment" for NLP, proving that a massive model pre-trained on the internet could solve almost any task.

6.1 The Problem with Static Vectors

In our earlier RNN example, we assigned a fixed vector to "I": $[1, 0]$. This approach is **Static**. But consider the word **"Bank"**:

- Sentence A: "I sat on the river **bank**."
- Sentence B: "I deposited money in the **bank**."

In a static model (like Word2Vec), "bank" has the exact same numbers in both sentences.

6.2 BERT's Solution: Contextual Embeddings

Because BERT uses **Self-Attention**, when it processes "bank" in Sentence A, the mechanism pulls information from "river" and "sat". In Sentence B, it pulls information from "money" and "deposited".

The result is a **Contextual Embedding**. The vector for "bank" changes based on its neighbors.

6.3 The Power of Fine-Tuning: The Swiss Army Knife

Before BERT, we used to build a specific model for every task (one for translation, one for spam detection, one for Q&A). BERT changed this paradigm.

Analogy: BERT is like a **Swiss Army Knife**.

1. **Pre-training (Making the Knife):** We train BERT on Wikipedia to understand language generally (using MLM and NSP). This is expensive and takes days.
2. **Fine-Tuning (Opening the right tool):** We take the pre-trained BERT and train it slightly on a specific task (e.g., Sentiment Analysis). This is cheap and takes minutes.

6.4 BERT's Training Objectives

How does BERT learn these smart embeddings? It doesn't just read books; it solves puzzles.

1. Masked Language Model (MLM): We hide 15% of the words in a sentence and ask BERT to guess them based on the context.

Input: "The [MASK] sat on the [MASK]."

Target: "cat", "mat"

Mathematically, BERT tries to maximize the probability of the correct word given the context: $P(w_i|w_1, \ldots, w_{i-1}, w_{i+1}, \ldots, w_n)$.

2. Next Sentence Prediction (NSP): We give BERT two sentences and ask: "Does sentence B logically follow sentence A?"

- A: "The man went to the store."
- B: "He bought a gallon of milk." $\rightarrow$ **True**
- B: "Penguins are flightless birds." $\rightarrow$ **False**

This teaches BERT to understand long-term relationships between sentences.

6.5 Bidirectionality: The Key Difference

Previous models (like the RNNs we studied) were usually **unidirectional**. They read from left to right. When the RNN reads "river", it hasn't seen "bank" yet.

BERT is **Bidirectional**. It uses the Transformer's power to look at the *entire sentence at once*. It doesn't read left-to-right; it reads everything simultaneously.

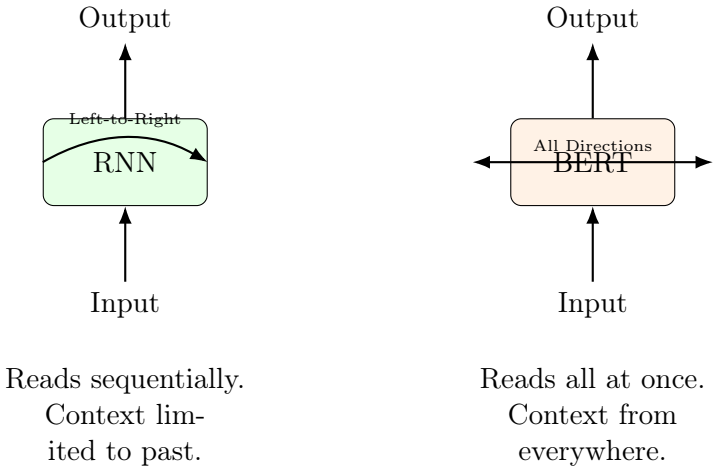

Figure 6: RNN vs BERT processing. BERT sees the future and the past simultaneously.

7 Conclusion

We started with a simple problem: translating a sentence. We built an **Encoder-Decoder** model using **RNNs** to handle sequences. We saw how plain RNNs forget things, so we upgraded to **LSTMs**. We then realized that squeezing everything into one vector is hard, so we added **Attention**. We then saw how **Transformers** replaced RNNs entirely with Self-Attention, and finally how **BERT** used this to create deep, contextual understanding of language.

The field of NLP is moving fast, but these mathematical foundations—vectors, matrices, gradients, and attention—remain the bedrock of modern AI.